🚀 Collections Framework Masterclass

Chapter 7 — Set Interface (Complete Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐⭐

The **Set** interface is one of the most frequently asked topics in Java interviews.

Interviewers often ask questions like:

- Why doesn't Set allow duplicates?
- How does HashSet detect duplicates?
- Difference between Set and List?
- Why must we override `equals()` and `hashCode()`?
- Which Set implementation should you choose?

Mastering the Set interface is essential before learning **HashSet**, **LinkedHashSet**, and **TreeSet**.

What is Set?

A **Set** is a collection that **does not allow duplicate elements**.

Package

```
java.util.Set
```

Declaration

```
public interface Set<E> extends Collection<E>
```

Hierarchy

```
                Iterable
                    ↑
               Collection
                    ↑
                  Set
          ___________|____________
```

```
        |           |           |
    HashSet   LinkedHashSet   TreeSet
```

Unlike `List`, Set focuses on **uniqueness**, not position.

Characteristics of Set

✅ No duplicate elements

✅ Stores unique objects

✅ Supports fast searching (implementation-dependent)

✅ Allows at most one `null` in most hash-based implementations

❌ No index-based access

❌ No positional methods like `get()`

Real-World Analogy

Imagine a company's Employee ID system.

```
EMP101

EMP102

EMP103

EMP101
```

Can the same employee ID exist twice?

No.

Employee IDs must be unique.

A Set works exactly like this.

Mathematical Set Concept

Java's Set follows mathematics.

Example

```
A = {1,2,3,4}

B = {3,4,5}
```

Notice

Duplicate values don't exist.

```
{1,2,2,3}
```

becomes

```
{1,2,3}
```

Why Doesn't Set Allow Duplicates?

Interview Favorite ⭐⭐⭐⭐⭐

Example

```
Set<Integer> set = new HashSet<>();

set.add(10);
set.add(20);
set.add(10);

System.out.println(set);
```

Output

```
[10,20]
```

The second `10` is ignored.

The exact order depends on the implementation.

How Does Set Know an Object is Duplicate?

This is one of the most important interview questions.

Java uses two methods.

```
hashCode()

equals()
```

Process

```
New Object

↓

hashCode()

↓

Same Hash?

↓

equals()

↓

Duplicate?

↓

Reject
```

We'll study this deeply in the HashSet chapter.

Duplicate Detection Example

```
String s1 = new String("Java");

String s2 = new String("Java");
```

Although

```
s1 != s2
```

their

```
equals()
```

returns

```
true
```

Therefore

HashSet stores only one element.

Set vs List

Interview Favorite ⭐⭐⭐⭐⭐

Feature	List	Set
Duplicates	Allowed	Not Allowed
Order	Maintains insertion order	Depends on implementation
Index	Yes	No
get(index)	Yes	No
add(index)	Yes	No
Unique Elements	No	Yes

Why No Index?

Suppose

```
HashSet<String> set =
new HashSet<>();
```

Internally

```
Java

Spring

AWS
```

There is no guaranteed ordering.

If there is no guaranteed order,

there cannot be an index.

Hence

```
set.get(0);
```

doesn't exist.

Major Set Implementations

There are three major implementations.

```
HashSet

LinkedHashSet

TreeSet
```

Each solves a different problem.

HashSet

Characteristics

- Uses HashMap internally
- Fastest implementation
- No ordering guarantee
- Allows one null element

Best for

```
Fast lookup

Fast insertion

Fast deletion
```

LinkedHashSet

Characteristics

- Preserves insertion order
- Uses LinkedHashMap internally
- Slightly slower than HashSet

Example

```
Input

10

30

20
```

Output

```
10

30
```

```
20
```

Order preserved.

TreeSet

Characteristics

- Stores elements in sorted order
- Uses Red-Black Tree internally
- Does not allow null elements (natural ordering)

Example

```
Input

40

10

30

20
```

Output

```
10

20

30

40
```

Automatically sorted.

Ordering in Different Sets

Interview Favorite ⭐⭐⭐⭐⭐

```
HashSet
```

Output

```
30

10

50

20
```

Random-looking order (not guaranteed).

```
LinkedHashSet
```

Output

```
30

10

50

20
```

Insertion order maintained.

```
TreeSet
```

Output

```
10

20

30
```

```
50
```

Sorted order.

Common Methods

Since Set extends Collection,

it inherits all Collection methods.

---

add()

```
set.add("Java");
```

Returns

```
true
```

if insertion succeeds.

Duplicate

```
set.add("Java");
```

Returns

```
false
```

Interview Favorite.

---

remove()

```
set.remove("Java");
```

Returns boolean.

---

contains()

```
set.contains("Spring");
```

Checks existence.

---

size()

Returns number of unique elements.

---

isEmpty()

Returns

```
true
```

or

```
false
```

---

clear()

Removes everything.

---

iterator()

Traverses Set.

Example

```
Iterator<String> itr =
set.iterator();

while(itr.hasNext())
{
```

```
    System.out.println(
        itr.next()
    );
}
```

Null Handling

Interview Favorite ⭐⭐⭐⭐

HashSet

```
set.add(null);

set.add(null);
```

Stores

```
One null
```

LinkedHashSet

Also stores

```
One null
```

TreeSet

Generally

```
Null not allowed
```

because comparisons are required for ordering.

---

equals() and Set

Suppose

```
Set<Integer> s1 =
Set.of(1,2,3);

Set<Integer> s2 =
Set.of(3,2,1);
```

Result

```
s1.equals(s2)
```

returns

```
true
```

Why?

Set compares

Elements

not

Order

Unlike List.

Interview Favorite ⭐⭐⭐⭐⭐

---

hashCode() in Set

Contract

```
Equal Sets

↓
```

```
Equal HashCodes
```

Order doesn't matter.

Only elements matter.

Performance

Implementation	add()	remove()	contains()
HashSet	O(1) Average	O(1) Average	O(1) Average
LinkedHashSet	O(1) Average	O(1) Average	O(1) Average
TreeSet	O(log n)	O(log n)	O(log n)

Choosing the Right Set

Use

```
HashSet
```

When

- Performance is most important
- Order doesn't matter

Use

```
LinkedHashSet
```

When

- Preserve insertion order
- Need uniqueness

Use

```
TreeSet
```

When

- Automatic sorting
- Range operations
- Ordered traversal

Common Mistakes

Mistake 1

Trying

```
set.get(0);
```

Not possible.

Mistake 2

Expecting HashSet to preserve insertion order.

Wrong.

Use LinkedHashSet.

Mistake 3

Using mutable objects inside HashSet without properly overriding `equals()` and `hashCode()`.

This can lead to incorrect behavior, including objects becoming difficult to find after mutation.

Mistake 4

Expecting TreeSet to accept all objects.

Elements must either

- Implement `Comparable`, or
- Be inserted using a `Comparator`.

Otherwise

```
ClassCastException
```

may occur.

Real-World Use Cases

Unique Email IDs

```
Set<String> emails;
```

Unique Usernames

```
Set<String> usernames;
```

Unique Roles

```
Set<Role> roles;
```

Removing Duplicate Values

```
List<String> names =
Arrays.asList(
"A",
"B",
"A",
"C"
);
```

```
Set<String> unique =
new HashSet<>(names);
```

Distinct Tags

```
Set<String> tags;
```

Frequently Asked Interview Questions

Q1. Difference between List and Set?

- List allows duplicates.
- Set stores only unique elements.

Q2. How does Set detect duplicates?

Using both

- `hashCode()`
- `equals()`

Q3. Why doesn't Set have get(index)?

Because a Set has no concept of positional indexing.

Q4. Which Set maintains insertion order?

`LinkedHashSet`.

Q5. Which Set automatically sorts elements?

`TreeSet`.

Q6. Which Set provides the best average performance?

`HashSet`.

Q7. Can Set contain null?

- `HashSet` → Yes (one null)
- `LinkedHashSet` → Yes (one null)
- `TreeSet` → Generally No (natural ordering)

Q8. Why is overriding equals() and hashCode() important?

Because hash-based collections rely on them to determine object equality and detect duplicates correctly.

Q9. Are two Sets equal if their insertion order differs?

Yes.

If they contain the same elements,

they are equal regardless of order.

Q10. Which Set should I choose?

Requirement	Recommended Implementation
Fast operations	HashSet
Preserve insertion order	LinkedHashSet
Sorted elements	TreeSet

Key Takeaways

- `Set` stores **unique elements** only.
- It extends the `Collection` interface.
- It has **no index-based operations**.
- Duplicate detection depends on `hashCode()` and `equals()`.
- `HashSet` provides the best average performance.
- `LinkedHashSet` preserves insertion order.

- `TreeSet` maintains sorted order using a Red-Black Tree.
- Choosing the correct Set implementation depends on whether you need performance, ordering, or sorting.

Next Chapter

🚀 Chapter 8 – HashSet (Complete Internal Working)

We'll cover:

- Internal Architecture
- Relationship with HashMap
- Hashing Process
- Bucket Structure
- Duplicate Detection
- `hashCode()` & `equals()` Flow
- Collision Handling
- Load Factor
- Resizing
- Performance Analysis
- Real-world Examples
- 25+ Interview Questions